```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import MinMaxScaler,LabelEncoder
from sklearn.impute import SimpleImputer
```

```
df=pd.read_csv('/content/heart.csv')
print('Dataset loaded successfully')
```

```
Dataset loaded successfully
```

```
df.shape
```

```
(918, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   Age             918 non-null    int64  
 1   Sex             918 non-null    object 
 2   ChestPainType   918 non-null    object 
 3   RestingBP       918 non-null    int64  
 4   Cholesterol     918 non-null    int64  
 5   FastingBS       918 non-null    int64  
 6   RestingECG      918 non-null    object 
 7   MaxHR           918 non-null    int64  
 8   ExerciseAngina  918 non-null    object 
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object 
 11  HeartDisease    918 non-null    int64  
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

```
df.describe()
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

```
df.isnull().sum()
```

	0
Age	0
Sex	0
ChestPainType	0
RestingBP	0
Cholesterol	0
FastingBS	0
RestingECG	0
MaxHR	0
ExerciseAngina	0
Oldpeak	0
ST_Slope	0
HeartDisease	0

dtype: int64

```
print("no.of duplicate rows:",df.duplicated().sum())
```

```
no.of duplicate rows: 0
```

```
#create Labelencoder
le=LabelEncoder()
#Apply labelencoding to the column sex
df['Sex']=le.fit_transform(df['Sex'])

print(df['Sex'].head())
```

```
0    1
1    0
2    1
3    0
4    1
Name: Sex, dtype: int64
```

```
# One-hot encoding specific categorical columns
df_encoded = pd.get_dummies(df, columns=['ChestPainType', 'RestingECG', 'ExerciseAngina'])
print(df_encoded)
```

```
     Age  Sex  RestingBP  Cholesterol  FastingBS  MaxHR  Oldpeak ST_Slope  \
0     40    1        140          289          0    172      0.0       Up
1     49    0        160          180          0    156      1.0     Flat
2     37    1        130          283          0     98      0.0       Up
3     48    0        138          214          0    108      1.5     Flat
4     54    1        150          195          0    122      0.0       Up
..   ...  ...        ...          ...        ...    ...      ...      ...
913   45    1        110          264          0    132      1.2     Flat
914   68    1        144          193          1    141      3.4     Flat
915   57    1        130          131          0    115      1.2     Flat
916   57    0        130          236          0    174      0.0     Flat
917   38    1        138          175          0    173      0.0       Up

     HeartDisease  ChestPainType_ASY  ChestPainType_ATA  ChestPainType_NAP  \
0               0              False               True              False
1               1              False              False               True
2               0              False               True              False
3               1               True              False              False
4               0              False              False               True
..            ...                ...                ...                ...
913             1              False              False              False
914             1               True              False              False
915             1               True              False              False
916             1              False               True              False
917             0              False              False               True

     ChestPainType_TA  RestingECG_LVH  RestingECG_Normal  RestingECG_ST  \
0               False           False               True          False
1               False           False               True          False
2               False           False              False           True
3               False           False               True          False
4               False           False               True          False
..                ...             ...                ...            ...
913              True           False               True          False
914             False           False               True          False
915             False           False               True          False
916             False            True              False          False
917             False           False               True          False
```

```
     ExerciseAngina_N  ExerciseAngina_Y
0                True             False
1                True             False
2                True             False
3               False              True
4                True             False
..                ...               ...
913              True             False
914              True             False
915             False              True
916              True             False
917              True             False

[918 rows x 18 columns]
```

```
print(df[['ChestPainType', 'RestingECG', 'ExerciseAngina']].head())
```

```
  ChestPainType RestingECG ExerciseAngina
0           ATA     Normal              N
1           NAP     Normal              N
2           ATA         ST              N
3           ASY     Normal              Y
4           NAP     Normal              N
```

```
#create min max scaler
Scaler=MinMaxScaler()
#Normalize age and MaxHR col
df[['Age','MaxHR','Cholesterol']]=Scaler.fit_transform(df[['Age','MaxHR','Cholesterol']])
print(df[['Age','MaxHR','Cholesterol']].head())
```

```
        Age     MaxHR  Cholesterol
0  0.244898  0.788732     0.479270
1  0.428571  0.676056     0.298507
2  0.183673  0.267606     0.469320
3  0.408163  0.338028     0.354892
4  0.530612  0.436620     0.323383
```

```
# Create a new feature 'CardioWorkload' as RestingBP + MaxHR
df['CardioWorkload'] = df['RestingBP'] + df['MaxHR']
print('CardioWorkload feature Created')
display(df[["RestingBP","MaxHR","CardioWorkload"]].head())
```

```
CardioWorkload feature Created
```

	RestingBP	MaxHR	CardioWorkload
0	140	0.788732	140.788732
1	160	0.676056	160.676056
2	130	0.267606	130.267606
3	138	0.338028	138.338028
4	150	0.436620	150.436620

```
# Create new feature 'HighRiskMetrics' where 1 indicates having multiple high risk factors
df['HighRiskMetrics'] = ((df['RestingBP'] > 140) & (df['Cholesterol'] > 240)).astype(int)
print('HighRiskMetrics feature Created')
print(df[['RestingBP', 'Cholesterol', 'HighRiskMetrics']].head())
```

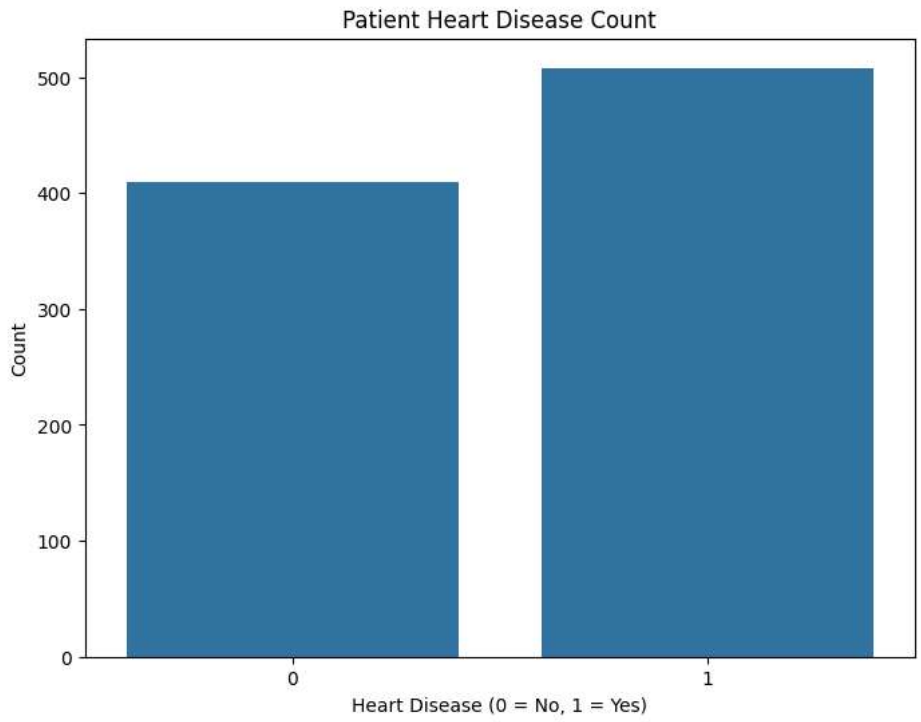
```
HighRiskMetrics feature Created
   RestingBP  Cholesterol  HighRiskMetrics
0        140     0.479270                0
1        160     0.298507                0
2        130     0.469320                0
3        138     0.354892                0
4        150     0.323383                0
```

```
# Add 'CardioWorkload' and 'IsHighRisk' features to the dataset
df['CardioWorkload'] = df['RestingBP'] + df['MaxHR']
df['IsHighRisk'] = (df['CardioWorkload'].apply(lambda x: 1 if x > 280 else 0))
print('CardioWorkload and IsHighRisk features are added to the dataset')
```

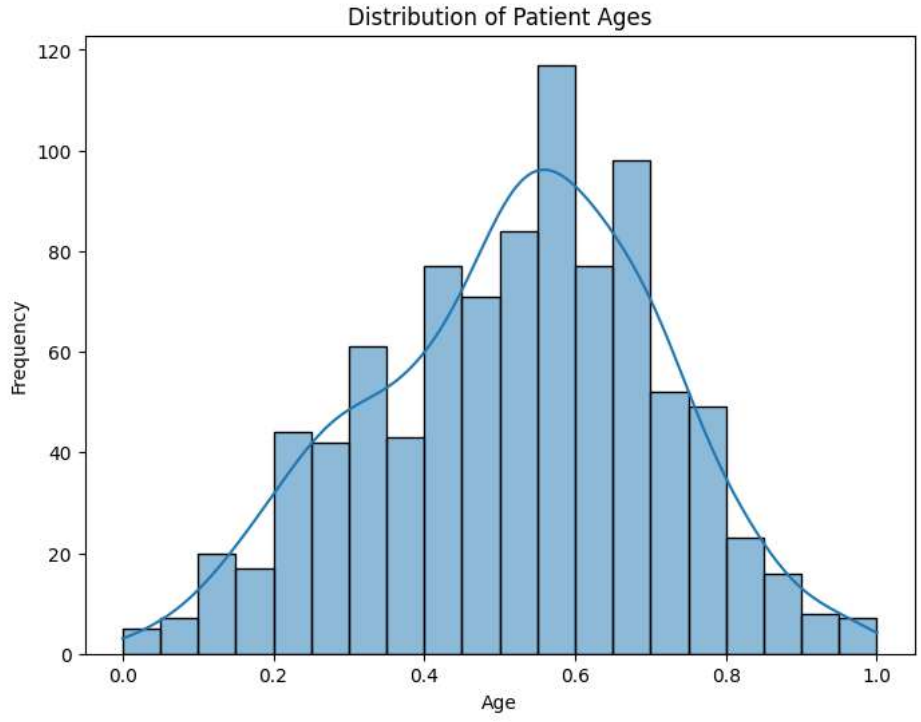
```
CardioWorkload and IsHighRisk features are added to the dataset
```

```
#(Univariate Visualization)plot the count of patients with heart disease Vs those without
plt.figure(figsize=(8,6))
sns.countplot(x='HeartDisease', data=df)
plt.title('Patient Heart Disease Count')
plt.xlabel('Heart Disease (0 = No, 1 = Yes)')
```

```
plt.ylabel('Count')
plt.show()
```




```
# Plot the distribution of patient ages
plt.figure(figsize=(8,6))
sns.histplot(df['Age'], bins=20, kde=True)
plt.title('Distribution of Patient Ages')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```




```
# Outlier detection for Cholesterol (Age rarely has outliers in this dataset)
Q1 = df['Cholesterol'].quantile(0.25)
Q3 = df['Cholesterol'].quantile(0.75)
IQR = Q3 - Q1
Lower_limit = Q1 - 1.5 * IQR
Upper_limit = Q3 + 1.5 * IQR
outliers = df[(df['Cholesterol'] < Lower_limit) | (df['Cholesterol'] > Upper_limit)]
print("Outliers in Cholesterol:\n", outliers)
```

```
Outliers in Cholesterol:
           Age  Sex ChestPainType  RestingBP  Cholesterol  FastingBS  \
28   0.510204    0           ATA        113     0.776119          0
30   0.510204    1           NAP        145     0.859038          0
69   0.326531    1           ASY        150     0.683250          0
76   0.081633    1           ASY        118     0.877280          0
103  0.244898    1           ASY        120     0.772803          1
..        ...  ...           ...        ...          ...        ...
535  0.571429    1           ASY        130     0.000000          0
536  0.693878    1           NAP        133     0.000000          1
616  0.795918    0           NAP        115     0.935323          0
667  0.755102    0           NAP        140     0.691542          1
796  0.571429    0           ASY        134     0.678275          0

    RestingECG     MaxHR ExerciseAngina  Oldpeak ST_Slope  HeartDisease  \
28      Normal  0.471831              N      0.0       Up             0
30      Normal  0.492958              N      0.0     Flat             1
69      Normal  0.774648              N      0.0       Up             0
76      Normal  0.492958              N      0.0     Flat             1
103     Normal  0.647887              Y      1.0     Flat             1
..         ...       ...            ...      ...      ...           ...
535        LVH  0.436620              Y      1.0     Flat             1
536         ST  0.415493              Y      1.2     Flat             1
616        LVH  0.704225              N      1.6     Flat             0
667        LVH  0.683099              N      0.8       Up             0
796        LVH  0.633803              Y      1.9     Flat             1

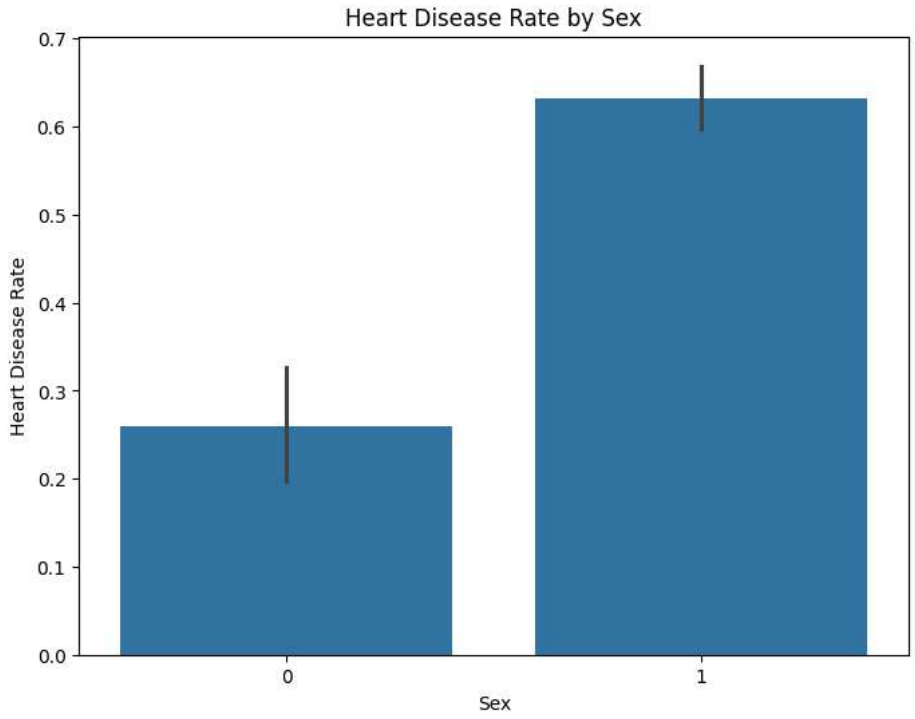
     CardioWorkload  HighRiskMetrics  IsHighRisk
28       113.471831                0           0
30       145.492958                0           0
69       150.774648                0           0
76       118.492958                0           0
103      120.647887                0           0
..              ...              ...         ...
535      130.436620                0           0
536      133.415493                0           0
616      115.704225                0           0
667      140.683099                0           0
796      134.633803                0           0

[183 rows x 15 columns]
```

```
print("DataSet Shape After removing outliers:",df.shape)
```

```
DataSet Shape After removing outliers: (918, 15)
```

```
# Bivariate Visualizations bar chart of heart disease rate by sex
plt.figure(figsize=(8,6))
sns.barplot(x='Sex', y='HeartDisease', data=df)
plt.title('Heart Disease Rate by Sex')
plt.xlabel('Sex')
plt.ylabel('Heart Disease Rate')
plt.show()
```



```
# drop PassengerId and restingECG (example of dropping unneeded columns in Heart dataset)
drop_columns = ['RestingECG'] # Change this to any columns you want to drop
df.drop(drop_columns, axis=1, inplace=True)
print('Columns dropped successfully')
```

```
Columns dropped successfully
```

```
print("After Preprocessing")
display(df.head(10))
```

```
After Preprocessing
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	0.244898	1	ATA	140	0.479270	0	0.788732	N	0.0	Up	0
1	0.428571	0	NAP	160	0.298507	0	0.676056	N	1.0	Flat	1
2	0.183673	1	ATA	130	0.469320	0	0.267606	N	0.0	Up	0
3	0.408163	0	ASY	138	0.354892	0	0.338028	Y	1.5	Flat	1
4	0.530612	1	NAP	150	0.323383	0	0.436620	N	0.0	Up	0
5	0.224490	1	NAP	120	0.562189	0	0.774648	N	0.0	Up	0
6	0.346939	0	ATA	130	0.393035	0	0.774648	N	0.0	Up	0
7	0.530612	1	ATA	110	0.344942	0	0.577465	N	0.0	Up	0
8	0.183673	1	ASY	140	0.343284	0	0.492958	Y	1.5	Flat	1
9	0.408163	0	ATA	120	0.470978	0	0.422535	N	0.0	Up	0

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv('/content/advertising.csv')
print('Dataset loaded successfully')
```

```
Dataset loaded successfully
```

```
df.shape
```

```
(200, 2)
```

```
df.head()
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```
df = df.drop(columns=['Radio', 'Newspaper'])
```

```
df.head()
```

	TV	Sales
0	230.1	22.1
1	44.5	10.4
2	17.2	12.0
3	151.5	16.5
4	180.8	17.9

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   TV      200 non-null    float64
 1   Sales   200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

```
df.describe()
```

	TV	Sales
count	200.000000	200.000000
mean	147.042500	15.130500
std	85.854236	5.283892
min	0.700000	1.600000
25%	74.375000	11.000000
50%	149.750000	16.000000
75%	218.825000	19.050000
max	296.400000	27.000000

```
df.isnull().sum()
```

	0
TV	0
Sales	0

dtype: int64

```
# 2. Apply your floor conversion logic to the 'TV' budget column instead
df['TV'] = df['TV'].apply(np.floor)
df
```

	TV	Sales
0	230.0	22.1
1	44.0	10.4
2	17.0	12.0
3	151.0	16.5
4	180.0	17.9
...	...	...
195	38.0	7.6
196	94.0	14.0
197	177.0	14.8
198	283.0	25.5
199	232.0	18.4

200 rows × 2 columns

```
X = df.iloc[:, :-1].values
X[:5]
```

```
array([[230.],
       [ 44.],
       [ 17.],
       [151.],
       [180.]])
```

```
# Dependent variable (selecting all rows, last column)
Y = df.iloc[:, -1].values
Y[:5]
```

```
array([22.1, 10.4, 12. , 16.5, 17.9])
```

```
#Split the dataset into training and testing
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)
```

```
#create the model
model=LinearRegression()
```

```
#train the model
model.fit(X_train,Y_train)
print("Model Trained")
```

```
Model Trained
```

```
#Generate predictions for training and testing datasets using the trained model
Y_train_pred=model.predict(X_train)
Y_test_pred=model.predict(X_test)
print("Predictions generate for both training and testing datasets")
```

```
Predictions generate for both training and testing datasets
```

```
#Evaluate the performance of the model by using metrics
print("Model Evaluation Metrics")
print("Mean Absolute Error (MAE):", mean_absolute_error(Y_train, Y_train_pred))
print("Mean Squared Error (MSE):", mean_squared_error(Y_train, Y_train_pred))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(Y_train, Y_train_pred)))
print("R-squared (R2) Score:", r2_score(Y_train, Y_train_pred))
```

```
Model Evaluation Metrics
Mean Absolute Error (MAE): 1.8012608958723155
Mean Squared Error (MSE): 5.004972524308184
```

```
Root Mean Squared Error (RMSE): 2.2371795914293924
R-squared (R2) Score: 0.8132429358050267
```

```
#Evaluate the performance of the model by using metrics
print("Model Evaluation Metrics")
print("Mean Absolute Error (MAE):", mean_absolute_error(Y_test, Y_test_pred))
print("Mean Squared Error (MSE):", mean_squared_error(Y_test, Y_test_pred))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(Y_test, Y_test_pred)))
print("R-squared (R2) Score:", r2_score(Y_test, Y_test_pred))
```

```
Model Evaluation Metrics
Mean Absolute Error (MAE): 1.949914506326006
Mean Squared Error (MSE): 6.103138675491602
Root Mean Squared Error (RMSE): 2.470453131612013
R-squared (R2) Score: 0.8024944524469484
```

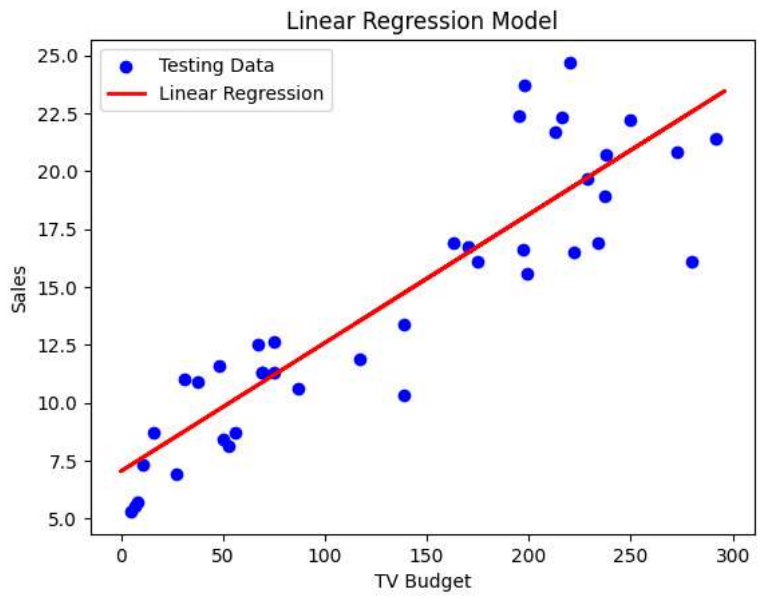
```
#slope and intercept
model.intercept_
```

```
np.float64(7.034008314297054)
```

```
model.coef_
```

```
array([0.05548185])
```

```
plt.scatter(X_test, Y_test, color='blue', label='Testing Data')
plt.plot(X_train, model.predict(X_train), color='red', linewidth=2, label='Linear Regression')
plt.xlabel('TV Budget')
plt.ylabel('Sales')
plt.title('Linear Regression Model')
plt.legend()
plt.show()
```



```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,mean_squared_error, r2_score
import matplotlib.pyplot as plt
```

```
df=pd.read_csv('/content/50_Startups_dataset.csv')
print("data loaded successfully")
```

```
data loaded successfully
```

```
df.head()
```

	Unnamed: 0	R&D Spend	Administration	Marketing Spend	State	Profit
0	0	165349.30	136897.90	471784.20	New York	192261.93
1	1	162597.80	151377.69	443898.63	California	191792.16
2	2	153441.61	101145.65	407934.64	Florida	191050.49
3	3	144372.51	118671.95	383199.72	New York	182902.09
4	4	142107.44	91391.87	366168.52	Florida	166188.04

```
df = df.drop(columns=['Unnamed: 0'])
```

```
df.shape
```

```
(50, 5)
```

```
df.head()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.30	136897.90	471784.20	New York	192261.93
1	162597.80	151377.69	443898.63	California	191792.16
2	153441.61	101145.65	407934.64	Florida	191050.49
3	144372.51	118671.95	383199.72	New York	182902.09
4	142107.44	91391.87	366168.52	Florida	166188.04

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object 
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

```
df.describe()
```

	R&D Spend	Administration	Marketing Spend	Profit
count	50.000000	50.000000	50.000000	50.000000
mean	73721.715600	121344.739600	211025.197800	112012.739200
std	45902.256482	28017.802755	122290.310726	40306.180338
min	0.100000	51283.240000	0.100000	14681.500000
25%	39936.470000	103730.975000	129300.232500	90139.002500
50%	73051.180000	122699.895000	212716.340000	107978.290000
75%	101602.900000	144842.280000	299469.185000	139766.077500
max	165349.300000	182645.660000	471784.200000	192261.930000

```
df.isnull().sum()
```

	0
R&D Spend	0
Administration	0
Marketing Spend	0
State	0
Profit	0

dtype: int64

```
print("no.of duplicate rows:",df.duplicated().sum())
```

```
no.of duplicate rows: 0
```

```
df['State'].value_counts()
```

State	count
New York	17
California	17
Florida	16

dtype: int64

```
# Convert State column into dummy numeric columns
df = pd.get_dummies(df, columns=['State'], drop_first=True)
```

```
df.columns
```

```
Index(['R&D Spend', 'Administration', 'Marketing Spend', 'Profit',
       'State_Florida', 'State_New York'],
      dtype='object')
```

```
# Create feature matrix X by dropping the target column 'Profit'
X = df.drop(['Profit'], axis=1)
print("Feature matrix X created by dropping the 'Profit' column")
```

```
Feature matrix X created by dropping the 'Profit' column
```

```
# Select target vector Y containing the Profit column
Y = df['Profit']
print("Target vector Y created by selecting 'Profit' column")
```

```
Target vector Y created by selecting 'Profit' column
```

```
X.head()
```

	R&D Spend	Administration	Marketing Spend	State_Florida	State_New York
0	165349.30	136897.90	471784.20	False	True
1	162597.80	151377.69	443898.63	False	False
2	153441.61	101145.65	407934.64	True	False
3	144372.51	118671.95	383199.72	False	True
4	142107.44	91391.87	366168.52	True	False

```
Y.head()
```

	Profit
0	192261.93
1	191792.16
2	191050.49
3	182902.09
4	166188.04

dtype: float64

```
#Split the dataset into training and testing
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2,random_state=42)
print("Dataset splitted into training and testing sets")
```

```
Dataset splitted into training and testing sets
```

```
#create the model
model=LinearRegression()
print("Linear Regression Model created")
```

```
Linear Regression Model created
```

```
#Train the model
model.fit(X_train,Y_train)
print("Model trained on the training data")
```

```
Model trained on the training data
```

```
#Generate predictions for training and testing data
Y_train_pred=model.predict(X_train)
Y_test_pred=model.predict(X_test)
print("Predictions generated for training and testing data")
```

```
Predictions generated for training and testing data
```

```
#Evaluate the model performance on training data using metrics
train_mae=mean_absolute_error(Y_train,Y_train_pred)
train_mse=mean_squared_error(Y_train,Y_train_pred)
train_rmse=np.sqrt(train_mse)
train_r2=r2_score(Y_train,Y_train_pred)
print("Model performance on training data:")
print("Mean Absolute Error:",train_mae)
print("Mean Squared Error:",train_mse)
print("Root Mean Squared Error:",train_rmse)
print("R2 Score:",train_r2)
```

```
Model performance on training data:
Mean Absolute Error: 6662.656240898315
Mean Squared Error: 79700060.08259319
Root Mean Squared Error: 8927.489013300055
R2 Score: 0.9537019995248526
```

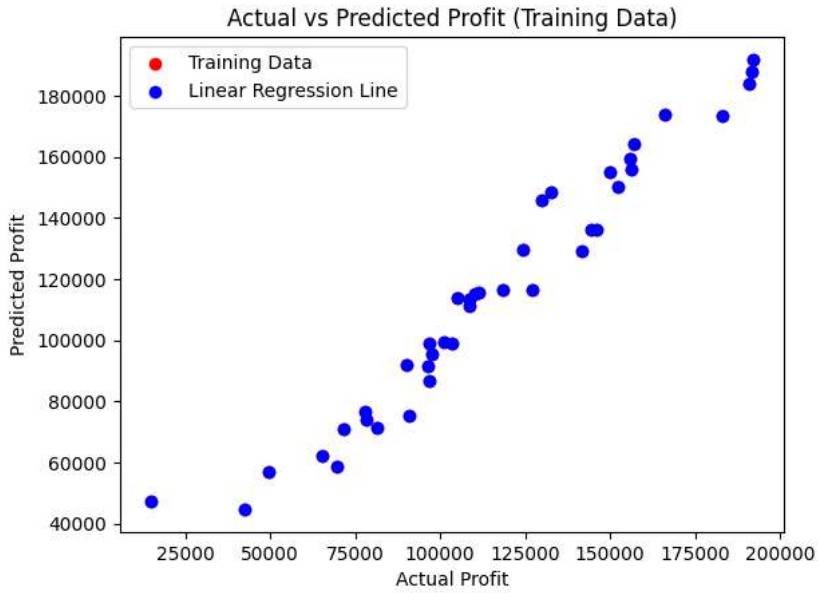
```
#Evaluate the model performance on testing data using metrics
test_mae=mean_absolute_error(Y_test,Y_test_pred)
test_mse=mean_squared_error(Y_test,Y_test_pred)
test_rmse=np.sqrt(test_mse)
test_r2=r2_score(Y_test,Y_test_pred)
print("Model performance on testing data:")
print("Mean Absolute Error:",test_mae)
print("Mean Squared Error:",test_mse)
print("Root Mean Squared Error:",test_rmse)
print("R2 Score:",test_r2)
```

```
Model performance on testing data:
Mean Absolute Error: 6961.477813252379
Mean Squared Error: 82010363.04430114
Root Mean Squared Error: 9055.957323458473
R2 Score: 0.8987266414328636
```

```
#coefficients and intercepts
print("Coefficients:",model.coef_)
print("Intercept:",model.intercept_)
```

```
Coefficients: [ 8.05630064e-01 -6.87878823e-02  2.98554429e-02  9.38793006e+02
  6.98775997e+00]
Intercept: 54028.06292388332
```

```
#Plot actual training datapoints and fitted linear regression line for visualisation
plt.scatter(Y_train,Y_train_pred,color='red',label='Training Data')
plt.scatter(Y_train,Y_train_pred,color='blue',label='Linear Regression Line')
plt.xlabel('Actual Profit')
plt.ylabel('Predicted Profit')
plt.title('Actual vs Predicted Profit (Training Data)')
plt.legend()
plt.show()
```



```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
# Load the dataset
df = pd.read_csv('/content/pH_Deviation_Plant_Stress_Polynomial.csv')
print("Dataset Loaded Successfully")
```

```
Dataset Loaded Successfully
```

```
df.shape
```

```
(101, 2)
```

Double-click (or enter) to edit

```
df.head()
```

	pH_Deviation	Plant_Stress
0	-5.0	82.22
1	-4.9	77.52
2	-4.8	77.68
3	-4.7	74.25
4	-4.6	74.09

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 101 entries, 0 to 100
Data columns (total 2 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   pH_Deviation  101 non-null    float64
 1   Plant_Stress  101 non-null    float64
dtypes: float64(2)
memory usage: 1.7 KB
```

```
df.describe()
```

	pH_Deviation	Plant_Stress
count	1.010000e+02	101.000000
mean	1.758769e-17	33.563267
std	2.930017e+00	23.024831
min	-5.000000e+00	2.590000
25%	-2.500000e+00	12.950000
50%	0.000000e+00	27.980000
75%	2.500000e+00	51.750000
max	5.000000e+00	83.580000

```
df.isnull().sum()
```

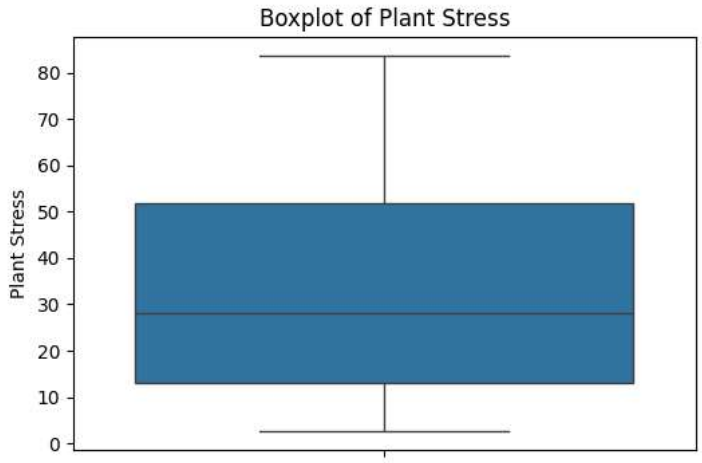
	0
pH_Deviation	0
Plant_Stress	0

dtype: int64

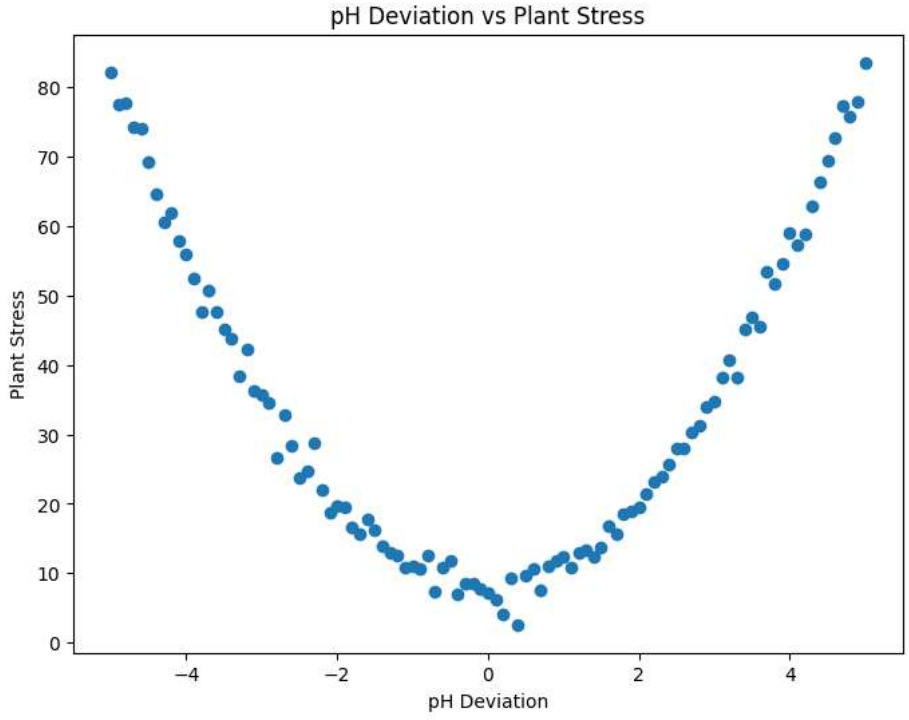
```
# Check duplicate records
print("Duplicate Records:", df.duplicated().sum())
```

```
Duplicate Records: 0
```

```
# Outlier detection
plt.figure(figsize=(6, 4))
sns.boxplot(y=df["Plant_Stress"])
plt.title("Boxplot of Plant Stress")
plt.ylabel("Plant Stress")
plt.show()
```




```
# Scatter plot
x = df["pH_Deviation"]
y = df["Plant_Stress"]
plt.figure(figsize=(8, 6))
plt.scatter(x, y)
plt.xlabel("pH Deviation")
plt.ylabel("Plant Stress")
plt.title("pH Deviation vs Plant Stress")
plt.show()
```




```
# Split the dataset into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(
x, y, test_size=0.2, random_state=42
)
```

```
print(f"Number of instances for training: {len(x_train)}")
print(f"Number of instances for testing: {len(x_test)}")
```

```
Number of instances for training: 80
Number of instances for testing: 21
```

```
poly_features=PolynomialFeatures(degree=2)
x_train_poly = poly_features.fit_transform(x_train.to_frame())
```

```
lr_model=LinearRegression()
lr_model.fit(x_train_poly,y_train)
```

▾ LinearRegression

LinearRegression()

```
model = LinearRegression()

model.fit(x_train_poly, y_train)
print("Model Trained Successfully")
```

```
Model Trained Successfully
```

```
x_test_poly=poly_features.transform(x_test.to_frame())
y_pred = lr_model.predict(x_test_poly)
df=pd.DataFrame({'Actual':y_test,'Predicted':y_pred})
x_test_poly = poly_features.transform(x_test.to_frame())
print(df)
```

```
    Actual  Predicted
84   45.21  42.867381
55    9.67   8.761611
66   16.85  15.754182
67   15.70  16.750118
45   11.76   8.709079
39   10.85  11.559547
22   26.62  31.373974
44   10.77   9.034053
10   55.87  55.807835
0    82.22  82.773943
18   42.26  38.557931
30   19.70  19.888046
97   77.34  74.547483
33   15.58  16.571507
77   30.26  30.011764
4    74.09  71.267003
93   62.89  63.719014
78   31.18  31.668157
12   47.72  51.135111
31   19.52  18.722492
76   27.98  28.415413
```

```
# Evaluate model performance on training data using MAE, MSE, RMSE, and R2 score
y_train_pred = model.predict(x_train_poly)
mae = mean_absolute_error(y_train, y_train_pred)
mse = mean_squared_error(y_train, y_train_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_train, y_train_pred)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R2 Score:", r2)
```

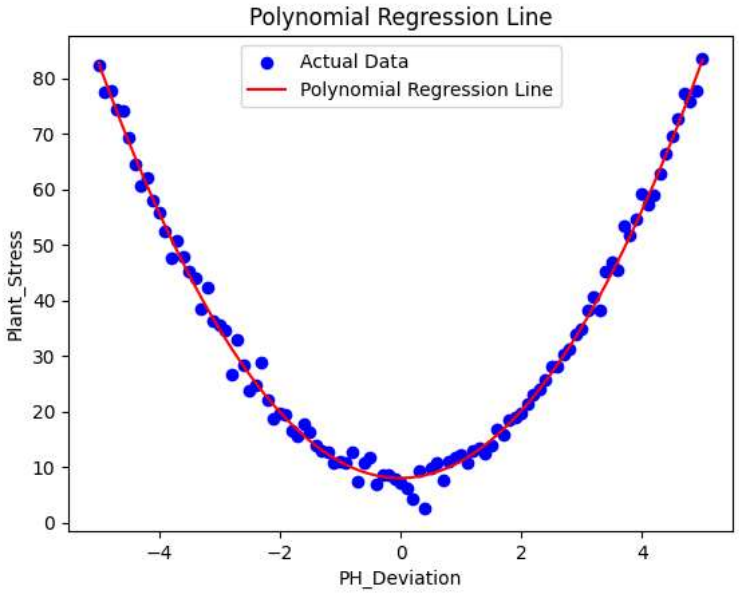
```
Mean Absolute Error (MAE): 1.2932269163635577
Mean Squared Error (MSE): 2.9896030734499712
Root MSE (RMSE): 1.7290468684943074
R2 Score: 0.9942808820129064
```

```
# Evaluate model performance on testing data using MAE, MSE, RMSE, and R2 score
x_test_poly = poly_features.transform(x_test.to_frame())
y_test_pred = model.predict(x_test_poly)
mae = mean_absolute_error(y_test, y_test_pred)
mse = mean_squared_error(y_test, y_test_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_test_pred)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root MSE (RMSE):", rmse)
print("R2 Score: ", r2)
```

```
Mean Absolute Error (MAE): 1.5701912386370716
Mean Squared Error (MSE): 4.205698200190934
```

```
Root MSE (RMSE): 2.0507799004746787
R2 Score:  0.9920746205603929
```

```
plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x, model.predict(poly_features.transform(x.to_frame())), color='red', label='Polynomial Regression Line')
plt.xlabel('PH_Deviation')
plt.ylabel('Plant_Stress')
plt.title('Polynomial Regression Line')
plt.legend()
plt.show()
```



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, LabelEncoder,MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix,roc_auc_score,roc_curve
```

```
df=pd.read_csv('/content/data.csv')
print("Dataset loaded successfully")
```

```
Dataset loaded successfully
```

```
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_m
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1

5 rows × 33 columns

```
df.shape
```

```
(569, 33)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       569 non-null    int64
 1   diagnosis                569 non-null    object
 2   radius_mean              569 non-null    float64
 3   texture_mean             569 non-null    float64
 4   perimeter_mean           569 non-null    float64
 5   area_mean                569 non-null    float64
 6   smoothness_mean          569 non-null    float64
 7   compactness_mean         569 non-null    float64
 8   concavity_mean           569 non-null    float64
 9   concave points_mean      569 non-null    float64
 10  symmetry_mean            569 non-null    float64
 11  fractal_dimension_mean   569 non-null    float64
 12  radius_se                569 non-null    float64
 13  texture_se               569 non-null    float64
 14  perimeter_se             569 non-null    float64
 15  area_se                  569 non-null    float64
 16  smoothness_se            569 non-null    float64
 17  compactness_se           569 non-null    float64
 18  concavity_se             569 non-null    float64
 19  concave points_se        569 non-null    float64
 20  symmetry_se              569 non-null    float64
 21  fractal_dimension_se     569 non-null    float64
 22  radius_worst             569 non-null    float64
 23  texture_worst            569 non-null    float64
 24  perimeter_worst          569 non-null    float64
 25  area_worst               569 non-null    float64
 26  smoothness_worst         569 non-null    float64
 27  compactness_worst        569 non-null    float64
 28  concavity_worst          569 non-null    float64
 29  concave points_worst     569 non-null    float64
 30  symmetry_worst           569 non-null    float64
 31  fractal_dimension_worst  569 non-null    float64
 32  Unnamed: 32              0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

```
df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mea
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.00000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.08879
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.07972
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.00000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.02956
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.06154
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.13070
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.42680

8 rows × 32 columns

```
print("number of duplicate records:",df.duplicated().sum())
```

```
number of duplicate records: 0
```

```
df.isnull().sum()
```

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0
radius_worst	0
texture_worst	0
perimeter_worst	0
area_worst	0
smoothness_worst	0
compactness_worst	0
concavity_worst	0
concave points_worst	0
symmetry_worst	0

```
# Drop unnecessary columns
df.drop(['id', 'Unnamed: 32'], axis=1, inplace=True, errors='ignore')
```

```
print(df.head())
```

```
dtype: int64
  diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0         M        17.99         10.38          122.80     1001.0
1         M        20.57         17.77          132.90     1326.0
2         M        19.69         21.25          130.00     1203.0
3         M        11.42         20.38           77.58      386.1
4         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  ...  radius_worst  texture_worst  perimeter_worst  \
0         0.2419  ...         25.38          17.33           184.60
1         0.1812  ...         24.99          23.41           158.80
2         0.2069  ...         23.57          25.53           152.50
3         0.2597  ...         14.91          26.50            98.87
4         0.1809  ...         22.54          16.67           152.20

   area_worst  smoothness_worst  compactness_worst  concavity_worst  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   concave points_worst  symmetry_worst  fractal_dimension_worst
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 31 columns]
```

```
# Label Encoding
le=LabelEncoder()
df['diagnosis']=le.fit_transform(df['diagnosis'])
df.head()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	c point
0	1	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0
1	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0
2	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0
3	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0
4	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0

5 rows × 31 columns

```
df.head()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	c point
0	1	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0
1	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0
2	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0
3	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0
4	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0

5 rows × 31 columns

```
#Outlier Detection
Q1=df['radius_mean'].quantile(0.25)
Q3=df['radius_mean'].quantile(0.75)

IQR = Q3 - Q1
print(IQR)
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
```

```
print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
```

```
4.08
Lower Limit: 5.579999999999999
Upper Limit: 21.9
```

```
#print outliers
outliers = df[(df['radius_mean'] < lower_limit) | (df['radius_mean'] > upper_limit)]
print(outliers)
```

```
     diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
82           1        25.22         24.91           171.5     1878.0
108          1        22.27         19.67           152.8     1509.0
122          1        24.25         20.20           166.2     1761.0
164          1        23.27         22.04           152.1     1686.0
180          1        27.22         21.87           182.1     2250.0
202          1        23.29         26.67           158.9     1685.0
212          1        28.11         18.47           188.5     2499.0
236          1        23.21         26.97           153.5     1670.0
339          1        23.51         24.27           155.1     1747.0
352          1        25.73         17.46           174.2     2010.0
369          1        22.01         21.90           147.2     1482.0
461          1        27.42         26.27           186.9     2501.0
503          1        23.09         19.83           152.1     1682.0
521          1        24.63         21.60           165.5     1841.0

     smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
82           0.10630            0.2665          0.3339              0.18450
108          0.13260            0.2768          0.4264              0.18230
122          0.14470            0.2867          0.4268              0.20120
164          0.08439            0.1145          0.1324              0.09702
180          0.10940            0.1914          0.2871              0.18780
202          0.11410            0.2084          0.3523              0.16200
212          0.11420            0.1516          0.3201              0.15950
236          0.09509            0.1682          0.1950              0.12370
339          0.10690            0.1283          0.2308              0.14100
352          0.11490            0.2363          0.3368              0.19130
369          0.10630            0.1954          0.2448              0.15010
461          0.10840            0.1988          0.3635              0.16890
503          0.09342            0.1275          0.1676              0.10030
521          0.10300            0.2106          0.2310              0.14710

     symmetry_mean  ...  radius_worst  texture_worst  perimeter_worst  \
82          0.1829  ...         30.00          33.62            211.7
108         0.2556  ...         28.40          28.01            206.8
122         0.2655  ...         26.02          23.99            180.9
164         0.1801  ...         28.01          28.22            184.2
180         0.1800  ...         33.12          32.85            220.8
202         0.2200  ...         25.12          32.68            177.0
212         0.1648  ...         28.11          18.47            188.5
236         0.1909  ...         31.01          34.51            206.0
339         0.1797  ...         30.67          30.73            202.4
352         0.1956  ...         33.13          23.58            229.3
369         0.1824  ...         27.66          25.80            195.0
461         0.2061  ...         36.04          31.37            251.2
503         0.1505  ...         30.79          23.87            211.5
521         0.1991  ...         29.92          26.93            205.7

     area_worst  smoothness_worst  compactness_worst  concavity_worst  \
82       2562.0            0.1573             0.6076           0.6476
108      2360.0            0.1701             0.6997           0.9608
122      2073.0            0.1696             0.4244           0.5803
164      2403.0            0.1228             0.3583           0.3948
180      3216.0            0.1472             0.4034           0.5340
202      1986.0            0.1536             0.4167           0.7892
212      2499.0            0.1142             0.1516           0.3201
236      2944.0            0.1481             0.4126           0.5820
339      2906.0            0.1515             0.2678           0.4819
```

```
#remove outliers
df = df[(df['radius_mean'] >= lower_limit) & (df['radius_mean'] <= upper_limit)]
print("Dataset Shape After Removing Outliers:", df.shape)
```

```
Dataset Shape After Removing Outliers: (555, 31)
```

```
#outlier detection for area_mean
Q1=df['area_mean'].quantile(0.25)
Q3=df['area_mean'].quantile(0.75)
IQR = Q3 - Q1
print(IQR)
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)
```

```
329.34999999999997
Lower Limit: -77.32499999999999
Upper Limit: 1240.0749999999998
```

```
outliers = df[(df['area_mean'] < lower_limit) | (df['area_mean'] > upper_limit)]
print(outliers)
```

```
     diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
1            1        20.57         17.77           132.9     1326.0
4            1        20.29         14.34           135.1     1297.0
18           1        19.81         22.15           130.0     1260.0
23           1        21.16         23.04           137.2     1404.0
78           1        20.18         23.97           143.7     1245.0
95           1        20.26         23.03           132.4     1264.0
181          1        21.09         26.57           142.7     1311.0
210          1        20.58         22.14           134.7     1290.0
233          1        20.51         27.81           134.4     1319.0
237          1        20.48         21.46           132.5     1306.0
250          1        20.94         23.56           138.9     1364.0
260          1        20.31         27.06           132.9     1288.0
265          1        20.73         31.12           135.7     1419.0
272          1        21.75         20.99           147.3     1491.0
302          1        20.09         23.86           134.7     1247.0
321          1        20.16         19.66           131.1     1274.0
323          1        20.34         21.51           135.9     1264.0
365          1        20.44         21.78           133.8     1293.0
368          1        21.71         17.25           140.9     1546.0
372          1        21.37         15.10           141.3     1386.0
373          1        20.64         17.35           134.8     1335.0
393          1        21.61         22.28           144.4     1407.0
432          1        20.18         19.54           133.8     1250.0
449          1        21.10         20.52           138.1     1384.0
499          1        20.59         21.24           137.8     1320.0
533          1        20.47         20.67           134.7     1299.0
535          1        20.55         20.86           137.8     1308.0
563          1        20.92         25.09           143.0     1347.0
564          1        21.56         22.39           142.0     1479.0
565          1        20.13         28.25           131.2     1261.0
567          1        20.60         29.33           140.1     1265.0

     smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
1            0.08474           0.07864         0.08690              0.07017
4            0.10030           0.13280         0.19800              0.10430
18           0.09831           0.10270         0.14790              0.09498
23           0.09428           0.10220         0.10970              0.08632
78           0.12860           0.34540         0.37540              0.16040
95           0.09078           0.13130         0.14650              0.08683
181          0.11410           0.28320         0.24870              0.14960
210          0.09090           0.13480         0.16400              0.09561
233          0.09159           0.10740         0.15540              0.08340
237          0.08355           0.08348         0.09042              0.06022
250          0.10070           0.16060         0.27120              0.13100
260          0.10000           0.10880         0.15190              0.09333
265          0.09469           0.11430         0.13670              0.08646
272          0.09401           0.19610         0.21950              0.10880
302          0.10800           0.18380         0.22830              0.12800
321          0.08020           0.08564         0.11550              0.07726
323          0.11700           0.18750         0.25650              0.15040
365          0.09150           0.11310         0.09799              0.07785
368          0.09384           0.08562         0.11680              0.08465
372          0.10010           0.15150         0.19320              0.12550
373          0.09446           0.10760         0.15270              0.08941
393          0.11670           0.20870         0.28100              0.15620
432          0.11330           0.14890         0.21330              0.12590
449          0.09684           0.11750         0.15720              0.11550
```

```
#remove outliers
df = df[(df['area_mean'] >= lower_limit) & (df['area_mean'] <= upper_limit)]
print("Dataset Shape After Removing Outliers:", df.shape)
```

```
Dataset Shape After Removing Outliers: (524, 31)
```

```
#Define features (x) and target (y)
x = df.drop(['diagnosis'], axis=1)
y = df['diagnosis']
```

```
#split the dataset into training and testing
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
scaler = StandardScaler()

x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)
```

```
print("First 5 rows of scaled x_train:")
print(x_train[:5])
```

```
First 5 rows of scaled x_train:
[[ 2.64917817e-01  2.06327291e+00  2.69968188e-01  1.58723607e-01
  -9.68812100e-01  1.35167609e-01  5.19542034e-01  8.35516496e-02
  -1.73162410e+00 -2.07350072e-01 -1.12096035e-01  2.20332545e+00
  -2.84645024e-01 -8.50991831e-02 -2.48424944e-01  2.21812005e-01
   2.53057408e-01 -1.97322549e-01 -1.12418645e+00 -5.95436637e-02
   8.98346243e-02  2.43037534e+00  5.24497101e-02 -1.39641764e-02
  -1.04228870e+00 -3.14094832e-04  2.66889047e-01 -3.93295116e-01
  -1.73751560e+00 -3.21157329e-01]
 [ 1.10109940e-01 -7.49997121e-01  1.55430603e-01 -4.78074886e-03
   3.90127552e-01  6.60528651e-01  2.74473600e-02  3.03158616e-01
  -5.13209778e-01  4.31804492e-01 -3.92015641e-01 -1.13124877e+00
  -3.91506674e-01 -3.50827189e-01 -7.92900619e-01 -2.18672732e-01
  -4.10669209e-01 -3.46280309e-01 -1.07598653e+00 -2.24175303e-01
   3.08454290e-01 -7.32461114e-01  3.55444271e-01  1.08379070e-01
   4.06913918e-01  7.47126109e-01  1.25452811e-01  5.69435585e-01
  -5.07658608e-01  1.06104517e+00]
 [-5.16493373e-01 -2.99780608e-01 -5.12345129e-01 -5.71294586e-01
   5.51993672e-01 -2.60341918e-03 -4.28966274e-01 -2.75926017e-01
   5.33865027e-01 -1.60546992e-01 -3.65356631e-01  8.87896114e-01
  -4.55046033e-01 -3.96254216e-01  3.81317676e-01 -4.63762957e-01
  -3.84005873e-01 -6.25246556e-01  1.88402029e-02 -7.01929864e-01
  -4.94039848e-01  6.14622642e-02 -4.97990408e-01 -5.30889668e-01
   4.99135903e-01 -4.37660379e-01 -4.58157765e-01 -7.28389228e-01
   3.01689340e-01 -7.15835759e-01]
 [ 1.83879790e+00 -3.46435169e-01  1.82324912e+00  1.98315287e+00
   4.26915306e-01  7.41185741e-01  1.10865877e+00  1.66997111e+00
   1.34072829e-01  6.23404601e-01  2.14355242e+00 -3.00296753e-01
   1.69807180e+00  2.56052261e+00  3.22418421e-01  4.30195169e-01
   4.38748499e-01  7.28720163e-01 -5.29261734e-01  5.38008212e-01
   1.95610006e+00 -4.07820380e-01  1.75776525e+00  2.09923734e+00
   4.11305441e-01  4.43729682e-01  4.99638503e-01  1.04985725e+00
  -6.46655321e-01  5.89087050e-01]
 [-1.26104555e+00 -9.08622629e-01 -1.28547383e+00 -1.10001345e+00
  -2.00683786e-01 -1.10629759e+00 -1.13094138e+00 -1.30106861e+00
  -3.57100444e-01 -3.25820369e-01  3.27777630e-01  1.36222208e-01
   1.30526924e-01 -2.27008391e-01  2.51949762e+00 -7.07159011e-01
  -9.18320084e-01 -1.46708859e+00  7.39084707e-01 -1.02225941e-01
  -9.44609646e-01 -6.81032879e-01 -1.00065061e+00 -8.40539412e-01
   8.89981457e-01 -9.42651341e-01 -1.21933285e+00 -1.62471879e+00
   9.75929006e-02 -3.75805111e-01]]
```

```
#Model Building
model= LogisticRegression()
model.fit(x_train,y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

```
#Predictions on test data
y_pred=model.predict(x_test)
```

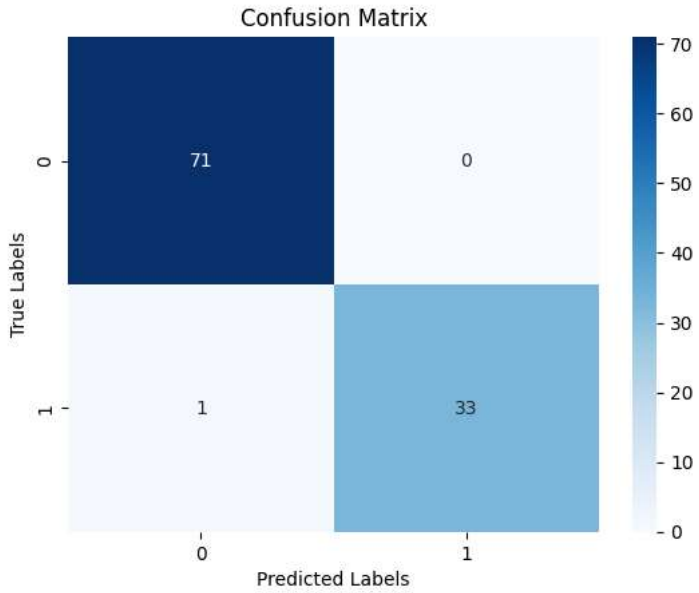
```
train_accuracy=model.score(x_train,y_train)
test_accuracy=accuracy_score(y_test,y_pred)
print("Training Accuracy:",train_accuracy)
print("Testing Accuracy:",test_accuracy)
```

```
Training Accuracy: 0.9856801909307876
Testing Accuracy: 0.9904761904761905
```

```
#confusion matrix
conf_matrix=confusion_matrix(y_test,y_pred)
print("Confusion Matrix:")
print(conf_matrix)
```

```
Confusion Matrix:
[[71  0]
 [ 1 33]]
```

```
#display confusion matrix
sns.heatmap(conf_matrix,annot=True,fmt='d',cmap='Blues')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.show()
```




```
#print classification report
class_report=classification_report(y_test,y_pred)
print("Classification Report:")
print(class_report)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.99      1.00      0.99        71
           1       1.00      0.97      0.99        34

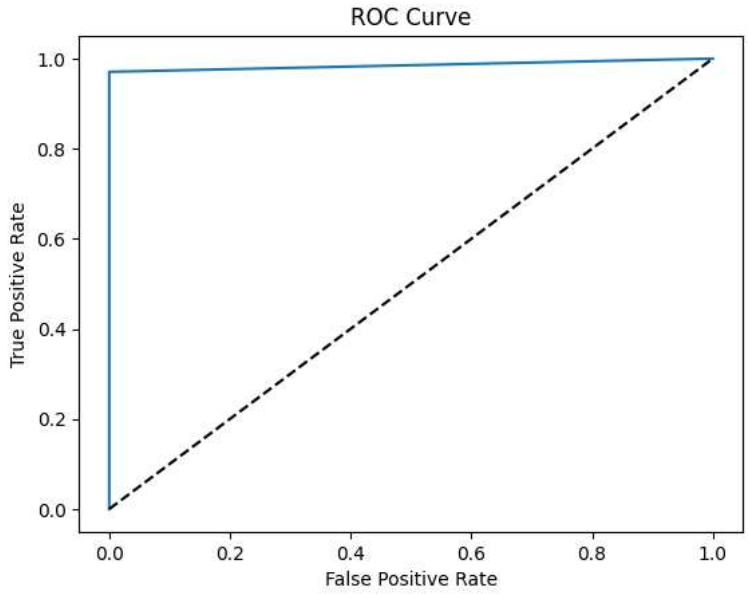
    accuracy                           0.99       105
   macro avg       0.99      0.99      0.99       105
weighted avg       0.99      0.99      0.99       105
```

```
#auc_roc_curve and auc_score
fpr,tpr,thresholds=roc_curve(y_test,y_pred)
auc_score=roc_auc_score(y_test,y_pred)
print("AUC Score:",auc_score)
```

```
AUC Score: 0.9852941176470589
```

```
#display auc_roc_curve
plt.plot(fpr,tpr,label='AUC={:.2f}'.format(auc_score))
plt.plot([0,1],[0,1],'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
```

```
Text(0.5, 1.0, 'ROC Curve')
```



```
#import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report,roc_curve,roc_auc_score
```

```
#Load the Dataset
df=pd.read_csv('/content/loan_approval_dataset.csv')
print("Dataset loaded successfully")
```

```
Dataset loaded successfully
```

```
df.shape
```

```
(4269, 13)
```

```
df.head()
```

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_asset
0	1	2	Graduate	No	9600000	29900000	12	778	2
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2
2	3	3	Graduate	No	9100000	29700000	20	506	1
3	4	3	Graduate	No	8200000	30700000	8	467	18
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4269 entries, 0 to 4268
Data columns (total 13 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   loan_id                   4269 non-null   int64
 1    no_of_dependents         4269 non-null   int64
 2    education                4269 non-null   object
 3    self_employed            4269 non-null   object
 4    income_annum             4269 non-null   int64
 5    loan_amount              4269 non-null   int64
 6    loan_term                4269 non-null   int64
 7    cibil_score              4269 non-null   int64
 8    residential_assets_value 4269 non-null   int64
 9    commercial_assets_value  4269 non-null   int64
 10   luxury_assets_value      4269 non-null   int64
 11   bank_asset_value         4269 non-null   int64
 12   loan_status              4269 non-null   object
dtypes: int64(10), object(3)
memory usage: 433.7+ KB
```

```
df.describe()
```

	loan_id	no_of_dependents	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commerc
count	4269.000000	4269.000000	4.269000e+03	4.269000e+03	4269.000000	4269.000000	4.269000e+03	
mean	2135.000000	2.498712	5.059124e+06	1.513345e+07	10.900445	599.936051	7.472617e+06	
std	1232.498479	1.695910	2.806840e+06	9.043363e+06	5.709187	172.430401	6.503637e+06	
min	1.000000	0.000000	2.000000e+05	3.000000e+05	2.000000	300.000000	-1.000000e+05	
25%	1068.000000	1.000000	2.700000e+06	7.700000e+06	6.000000	453.000000	2.200000e+06	
50%	2135.000000	3.000000	5.100000e+06	1.450000e+07	10.000000	600.000000	5.600000e+06	
75%	3202.000000	4.000000	7.500000e+06	2.150000e+07	16.000000	748.000000	1.130000e+07	
max	4269.000000	5.000000	9.900000e+06	3.950000e+07	20.000000	900.000000	2.910000e+07	

```
print("No of duplicate record",df.duplicated().sum())
```

```
No of duplicate record 0
```

```
print(df.isnull().sum())
```

```
loan_id                     0
 no_of_dependents           0
 education                  0
 self_employed              0
 income_annum               0
 loan_amount                0
 loan_term                  0
 cibil_score                0
 residential_assets_value   0
 commercial_assets_value    0
 luxury_assets_value        0
 bank_asset_value           0
 loan_status                0
dtype: int64
```

```
#feature summary
numerical_features=df.select_dtypes(include=['int64','float64']).columns
categorical_features=df.select_dtypes(include=['object']).columns
print("Faeture Summary")
print("numerical Features")
for col in numerical_features:
  print("-",col)
print("categorical Features")
for col in categorical_features:
  print("-",col)
```

```
Faeture Summary
numerical Features
- loan_id
-  no_of_dependents
-  income_annum
-  loan_amount
-  loan_term
-  cibil_score
-  residential_assets_value
-  commercial_assets_value
-  luxury_assets_value
-  bank_asset_value
categorical Features
-  education
-  self_employed
-  loan_status
```

```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df[' education']=le.fit_transform(df[' education'])
df[' self_employed']=le.fit_transform(df[' self_employed'])
```

```
#Target column encoding
df[' loan_status']=le.fit_transform(df[' loan_status'])
# 'Approved':1
# 'Rejected':0
```

```
#seperate Features and Target
x=df.iloc[:,:-1]
y=df.iloc[:,-1]
```

```
#Split the dataset into training and testing
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
#Model building
model=DecisionTreeClassifier(criterion='entropy')
model.fit(x_train,y_train)
```

```
▾       DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy')
```

```
#Predictions on test data
y_pred=model.predict(x_test)
```

```
#Evaluation metrics
accuracy=accuracy_score(y_test,y_pred)
conf_matrix=confusion_matrix(y_test,y_pred)
classification_rep=classification_report(y_test,y_pred)
```

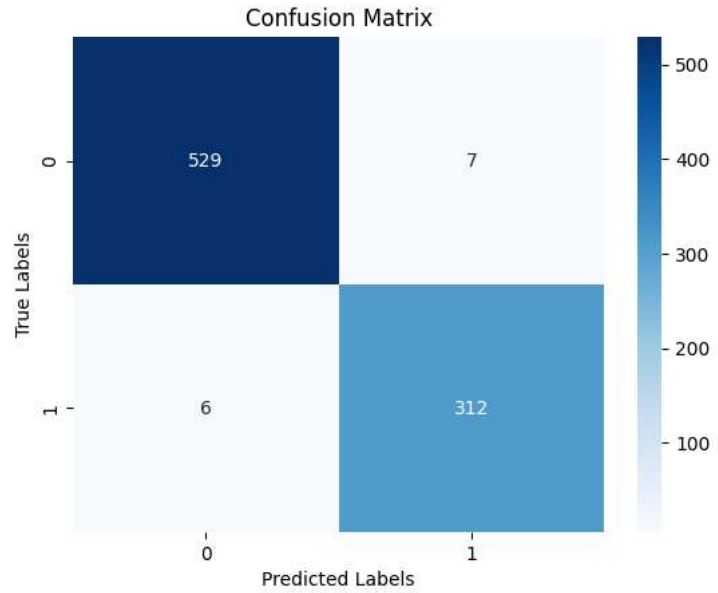
```
print("Accuracy:",accuracy)
print("Confusion Matrix:")
print(conf_matrix)
print("Classification Report:")
print(classification_rep)
```

```
Accuracy: 0.9847775175644028
Confusion Matrix:
[[529   7]
 [  6 312]]
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       536
           1       0.98      0.98      0.98       318

    accuracy                           0.98       854
   macro avg       0.98      0.98      0.98       854
weighted avg       0.98      0.98      0.98       854
```

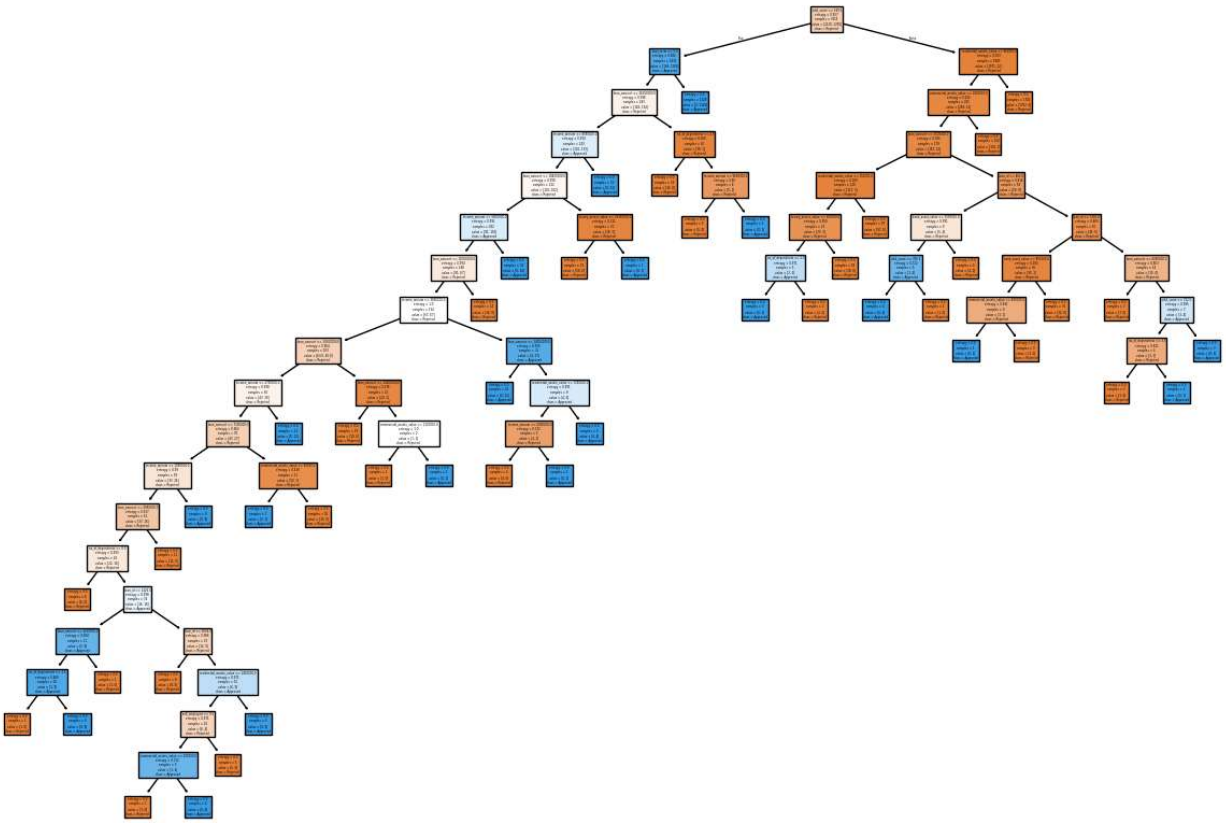
```
import seaborn as sns
#display confusion matrix
sns.heatmap(conf_matrix,annot=True,fmt='d',cmap='Blues')
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.show()
```



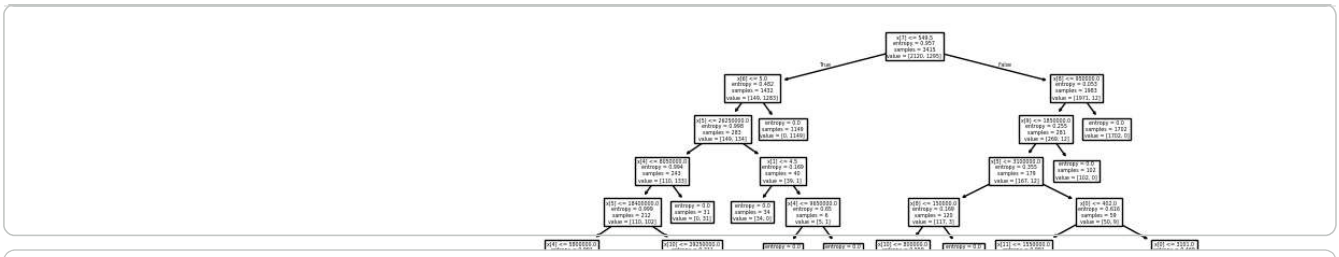

```
#training accuracy
y_train_pred=model.predict(x_train)
train_accuracy=accuracy_score(y_train,y_train_pred)
print("Training Accuracy:",train_accuracy)
#testing accuracy
y_test_pred=model.predict(x_test)
test_accuracy=accuracy_score(y_test,y_test_pred)
print("Testing Accuracy:",test_accuracy)
```

```
Training Accuracy: 1.0
Testing Accuracy: 0.9847775175644028
```

```
#display DecisionTree
from sklearn.tree import plot_tree
plt.figure(figsize=(15,10))
plot_tree(model,filled=True,feature_names=x.columns,class_names=['Rejected','Approved'])
plt.show()
```



```
#display decision tree
from sklearn import tree
plt.figure(figsize=(15,10))
tree.plot_tree(model)
plt.show()
```

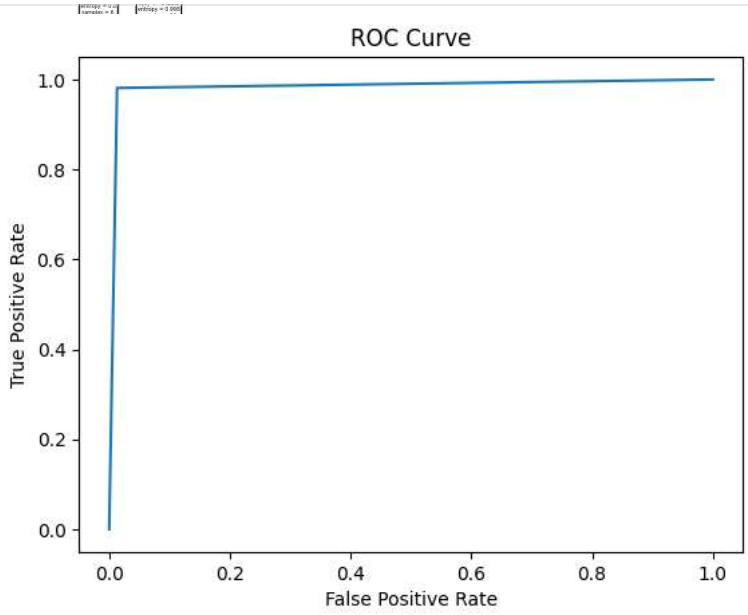


```
#auc
auc=roc_auc_score(y_test,y_pred)
print("AUC Score:",auc)
```

```
AUC Score: 0.9840361869895804
```



```
#roc_curve
fpr,tpr,thresholds=roc_curve(y_test,y_pred)
plt.plot(fpr,tpr)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.show()
```



Double-click (or enter) to edit

```
#import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report
```

```
df=pd.read_csv('/content/iris (2).csv')
print("Dataset loaded succefully")
```

```
Dataset loaded succefully
```

```
df.shape
```

```
(150, 5)
```

```
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
print("No.of records:",df.duplicated().sum())
```

```
No.of records: 3
```

```
#remove duplicate records
df.drop_duplicates(inplace=True)
df.shape
```

```
(147, 5)
```

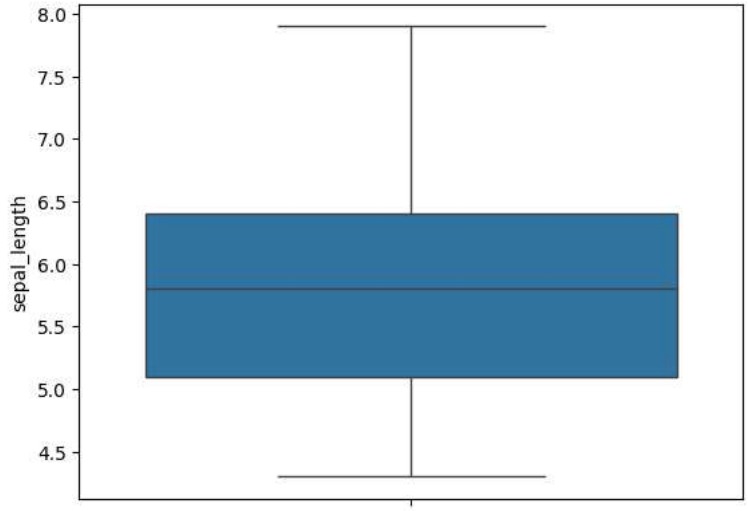
```
df.isnull().sum()
```

	0
sepal_length	0
sepal_width	0
petal_length	0
petal_width	0
species	0

dtype: int64

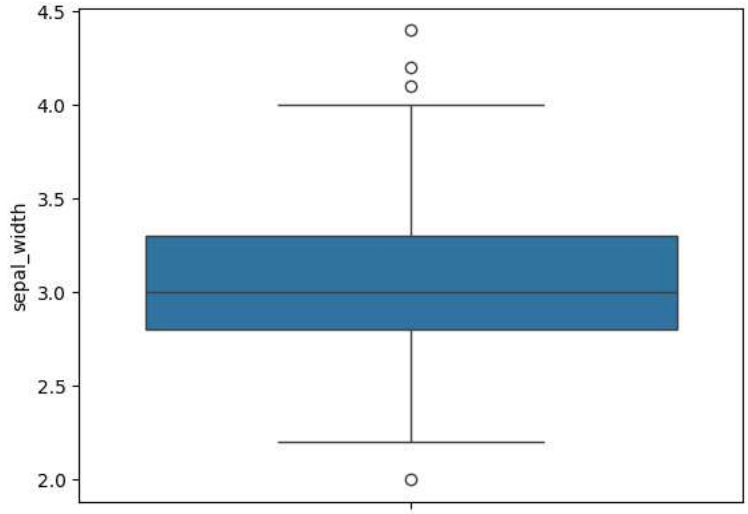
```
#identify outliers
sns.boxplot(df['sepal_length'])
```

```
<Axes: ylabel='sepal_length'>
```




```
#identfy outliers in sepal width
sns.boxplot(df['sepal_width'])
```

```
<Axes: ylabel='sepal_width'>
```

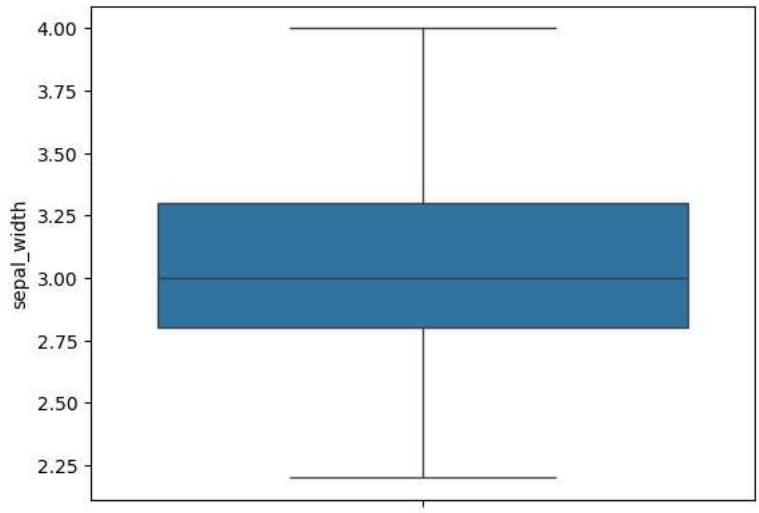



```
#remove outliers in sepal width
Q1=df['sepal_width'].quantile(0.25)
Q3=df['sepal_width'].quantile(0.75)
IQR=Q3-Q1
lower_limit=Q1-1.5*IQR
upper_limit=Q3+1.5*IQR
df=df[(df['sepal_width']>=lower_limit)&(df['sepal_width']<=upper_limit)]
df.shape
```

```
(143, 5)
```

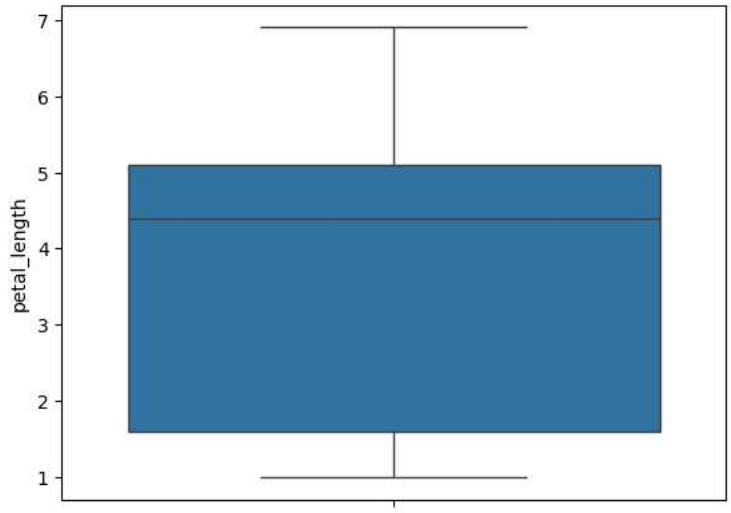
```
#outliers in sepal width
sns.boxplot(df['sepal_width'])
```

```
<Axes: ylabel='sepal_width'>
```



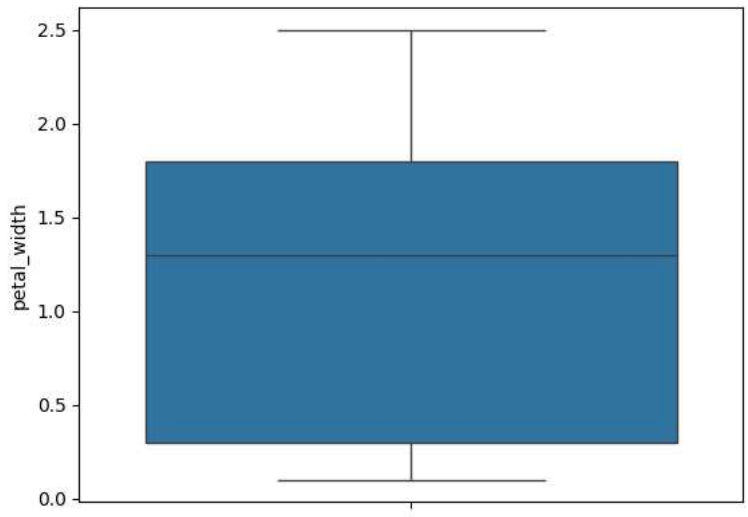

```
#identify petal length
sns.boxplot(df['petal_length'])
```

```
<Axes: ylabel='petal_length'>
```




```
#identify petal width
sns.boxplot(df['petal_width'])
```

```
<Axes: ylabel='petal_width'>
```




```
#label encoding for target
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df['species']=le.fit_transform(df['species'])
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
x=df.iloc[:,:-1]
y=df.iloc[:,-1]
```

```
#split the data into training and testing
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.20,random_state=15)
```

```
#standardization
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
x_train=sc.fit_transform(x_train)
x_test=sc.transform(x_test)
```

```
#model building
knn=KNeighborsClassifier(n_neighbors=5)
knn.fit(x_train,y_train)
```

```
▾ KNeighborsClassifier
KNeighborsClassifier()
```

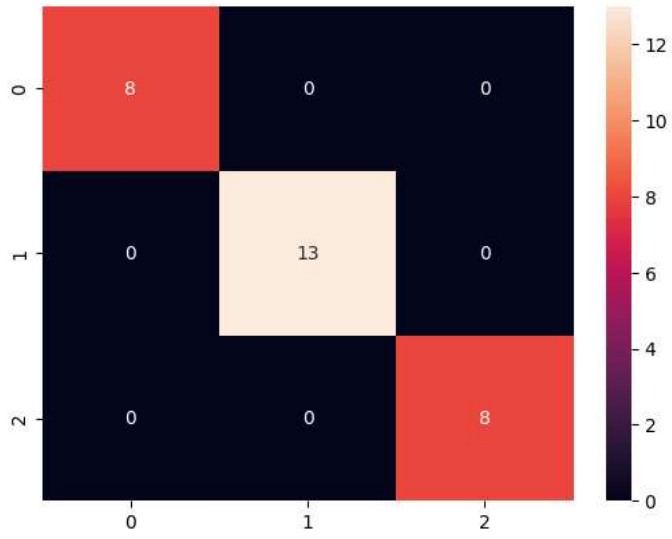
```
#predictions on test data
y_pred=knn.predict(x_test)
```

```
#evaluation metrics
accuracy_score(y_test,y_pred)
classification_report(y_test,y_pred)
confusion_matrix(y_test,y_pred)
```

```
array([[ 8,  0,  0],
       [ 0, 13,  0],
       [ 0,  0,  8]])
```

```
#confusion matrix
sns.heatmap(confusion_matrix(y_test,y_pred),annot=True)
```

```
<Axes: >
```

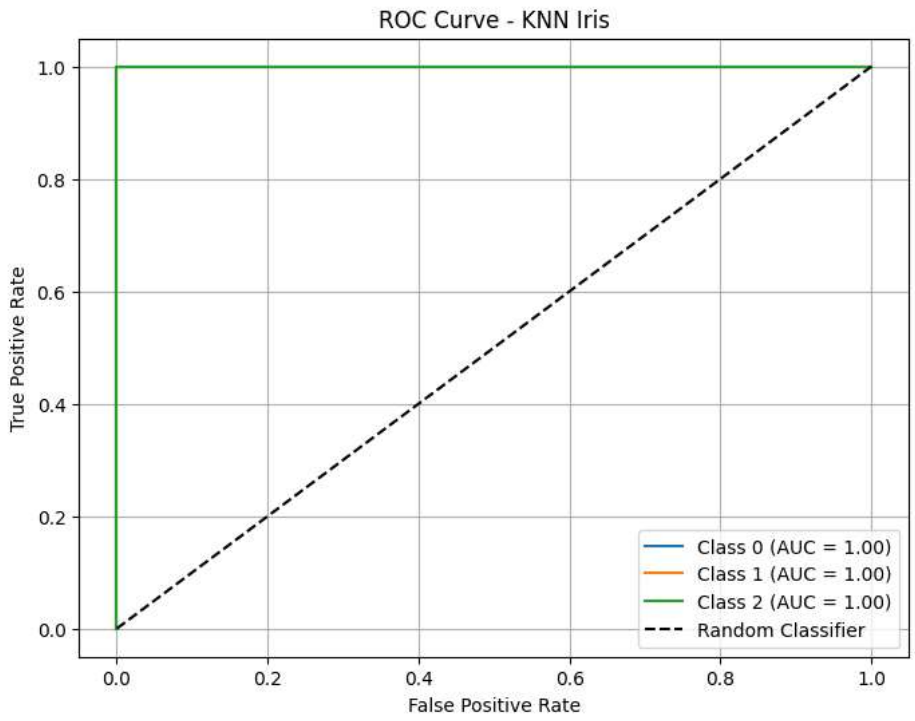



```
from sklearn.metrics import accuracy_score, roc_auc_score
y_pred = knn.predict(x_test)
y_proba = knn.predict_proba(x_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
roc_auc = roc_auc_score(
y_test,
y_proba,
```

```
multi_class='ovr'
)
print("ROC-AUC Score:", roc_auc)
```

```
Accuracy: 1.0
ROC-AUC Score: 1.0
```

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt
# Predict probabilities
y_proba = knn.predict_proba(x_test)
# Convert test labels into binary form
y_test_bin = label_binarize(y_test, classes=[0, 1, 2])
# Plot ROC curve for each class
plt.figure(figsize=(8, 6))
for i in range(3):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
    auc_score = roc_auc_score(y_test_bin[:, i], y_proba[:, i])
    plt.plot(
        fpr, tpr,
        label=f'Class {i} (AUC = {auc_score:.2f})'
    )
# Random classifier
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - KNN Iris')
plt.legend()
plt.grid()
plt.show()
```




```
overall_auc = roc_auc_score(
y_test_bin,

y_proba,
multi_class='ovr'
)
print("Overall ROC-AUC:", overall_auc)
```

```
Overall ROC-AUC: 1.0
```

```
#import libraries
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score,classification_report,confusion_matrix
```

```
#load the dataset
df=pd.read_csv('/content/Social_Network_Ads.csv')
```

```
df.head()
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

```
df.shape
```

```
(400, 5)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   User ID          400 non-null    int64
 1   Gender           400 non-null    object
 2   Age              400 non-null    int64
 3   EstimatedSalary  400 non-null    int64
 4   Purchased        400 non-null    int64
dtypes: int64(4), object(1)
memory usage: 15.8+ KB
```

```
df.describe()
```

	User ID	Age	EstimatedSalary	Purchased
count	4.000000e+02	400.000000	400.000000	400.000000
mean	1.569154e+07	37.655000	69742.500000	0.357500
std	7.165832e+04	10.482877	34096.960282	0.479864
min	1.556669e+07	18.000000	15000.000000	0.000000
25%	1.562676e+07	29.750000	43000.000000	0.000000
50%	1.569434e+07	37.000000	70000.000000	0.000000
75%	1.575036e+07	46.000000	88000.000000	1.000000
max	1.581524e+07	60.000000	150000.000000	1.000000

```
#print numerical and categorical variables
# Numerical variables
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
# Categorical variables
categorical_cols = df.select_dtypes(include=['object', 'category']).columns
print("Numerical Variables:")
print(list(numerical_cols))
print("\nCategorical Variables:")
print(list(categorical_cols))
```

```
Numerical Variables:
['User ID', 'Age', 'EstimatedSalary', 'Purchased']

Categorical Variables:
['Gender']
```

```
#identify duplicate rows
print("no of duplicate rows:",df.duplicated().sum())
```

```
no of duplicate rows: 0
```

```
#identify null values
df.isnull().sum()
```

	0
User ID	0
Gender	0
Age	0
EstimatedSalary	0
Purchased	0

dtype: int64

```
#seperate features and target
X = df.drop(columns=['User ID', 'Purchased'])
y = df['Purchased']
```

```
#Identify categorical columns in the entire features dataset (X)
categorical_cols_X = X.select_dtypes(include=['object', 'category']).columns
#Apply One-Hot Encoding to X before splitting to ensure identical columns
X = pd.get_dummies(X, columns=categorical_cols_X, drop_first=True)
```

```
#Split the data safely into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
#Standard Scaling
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(X_test)
```

```
from sklearn.svm import LinearSVC
#model buiding
model=LinearSVC(random_state=42)
model.fit(X_train,y_train)
```

```
LinearSVC
LinearSVC(random_state=42)
```

```
#Predictions
y_pred=model.predict(X_test)
accuracy=accuracy_score(y_test,y_pred)
print("Accuracy:",accuracy)
```

```
Accuracy: 0.8875
```

```
#Evaluation metrics,confusion matrix,TP,FP,TN,FN
confusion_mat=confusion_matrix(y_test,y_pred)
print("Confusion Matrix:")
print(confusion_mat)
```

```
Confusion Matrix:
[[50  2]
 [ 7 21]]
```

```
#training accuracy
training_accuracy=model.score(X_train,y_train)
print("Training Accuracy:",training_accuracy)
#testing accuracy
testing_accuracy=model.score(X_test,y_test)
print("Testing Accuracy:",testing_accuracy)
```

```
Training Accuracy: 0.81875
Testing Accuracy: 0.8875
```

```
#classification Report
classification_rep=classification_report(y_test,y_pred)
```

```
print("Classification Report:")
print(classification_rep)
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.96      0.92        52
           1       0.91      0.75      0.82        28

    accuracy                           0.89        80
   macro avg       0.90      0.86      0.87        80
weighted avg       0.89      0.89      0.88        80
```

```
# 1. Import libraries
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Load MNIST dataset
mnist = fetch_openml('mnist_784')
x=mnist.data
y=mnist.target.astype(int)
```

```
print("dataset shape:",x.shape)
```

```
dataset shape: (70000, 784)
```

```
#train test split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
```

```
# 4. Standardization
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

```
#simple perception
perceptron=Perceptron()
perceptron.fit(x_train_scaled,y_train)
y_pred=perceptron.predict(x_test_scaled)
accuracy=accuracy_score(y_test,y_pred)
print("Simple Perceptron Accuracy:",accuracy)
```

```
Simple Perceptron Accuracy: 0.8759285714285714
```

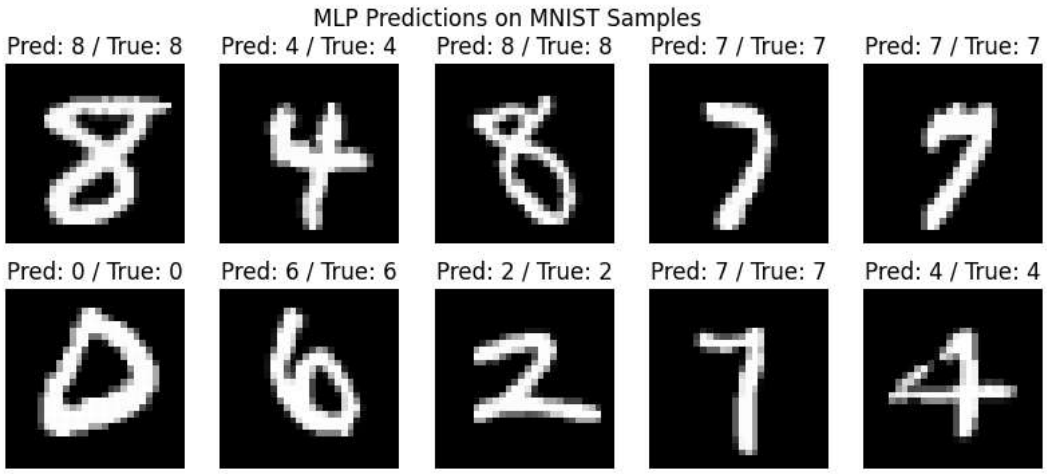
```
#Multi layer perception
mlp = MLPClassifier(hidden_layer_sizes=(100,),max_iter=20,random_state=42)
# Model training
mlp.fit(x_train_scaled, y_train)
# Prediction
y_pred_mlp= mlp.predict(x_test_scaled)
# Accuracy
mlp_accuracy=accuracy_score(y_test, y_pred_mlp)
print("Multi Layer Perceptron Accuracy:",mlp_accuracy)
```

```
Multi Layer Perceptron Accuracy: 0.9729285714285715
/usr/local/lib/python3.13/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic
  warnings.warn(
```

```
# 7. Comparison
print("Performance Comparison")
print("Simple Perceptron Accuracy: {:.2f}%".format(accuracy * 100))
print("MLP Accuracy : {:.2f}%".format(mlp_accuracy * 100))
```

```
Performance Comparison
Simple Perceptron Accuracy: 87.59%
MLP Accuracy : 97.29%
```

```
#8.Visualization
plt.figure(figsize=(10,4))
for i in range(10):
  plt.subplot(2,5,i+1)
  plt.imshow(x_test.iloc[i].values.reshape(28,28), cmap='gray')
  plt.title(f"Pred: {y_pred_mlp[i]} / True: {y_test.iloc[i]}")
  plt.axis('off')
plt.suptitle("MLP Predictions on MNIST Samples")
plt.figure(figsize=(10,4))
plt.show()
```




```
<Figure size 1000x400 with 0 Axes>
```

```
# Import libraries
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```
#Load MNIST dataset
mnist = fetch_openml('mnist_784')
x=mnist.data
y=mnist.target.astype(int)
print("dataset shape:",x.shape)
```

```
# 2. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# 3. Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# 4. ANN / MLP
ann = MLPClassifier(hidden_layer_sizes=(100,),max_iter=50,random_state=42)
ann.fit(X_train, y_train)
y_pred_ann = ann.predict(X_test)
```

```
# 5. SVM
svm = SVC(kernel='linear',random_state=42)
svm.fit(X_train, y_train)
y_pred_svm = svm.predict(X_test)
```

```
# 6. KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
```

```
def evaluate_model(name, y_test, y_pred):
  accuracy = accuracy_score(y_test, y_pred)
  precision = precision_score(y_test, y_pred,average='weighted')
  recall = recall_score(y_test, y_pred,average='weighted')
  f1 = f1_score(y_test, y_pred,average='weighted')
  print("\n", name)
  print("-------------------------")
  print("Accuracy :", round(accuracy, 4))
  print("Precision:", round(precision, 4))
  print("Recall :", round(recall, 4))
  print("F1-Score :", round(f1, 4))
```

```
# 8. Compare models
evaluate_model("ANN (MLP)", y_test, y_pred_ann)
evaluate_model("SVM", y_test, y_pred_svm)
evaluate_model("KNN", y_test, y_pred_knn)
```

```
 ANN (MLP)
-------------------------
Accuracy : 0.97
Precision: 0.9701
Recall : 0.97
F1-Score : 0.97

 SVM
-------------------------
Accuracy : 0.921
Precision: 0.9213
Recall : 0.921
F1-Score : 0.9209

 KNN
-------------------------
Accuracy : 0.9458
Precision: 0.9461
Recall : 0.9458
F1-Score : 0.9457
```

```
Start coding or generate with AI.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.metrics import davies_bouldin_score
```

```
df=pd.read_csv('/content/Mall_Customers.csv')
print("Data loaded succefully")
```

```
Data loaded succefully
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
df.shape
```

```
(200, 5)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.describe()
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
df.isnull().sum()
```

	0
CustomerID	0
Gender	0
Age	0
Annual Income (k$)	0
Spending Score (1-100)	0

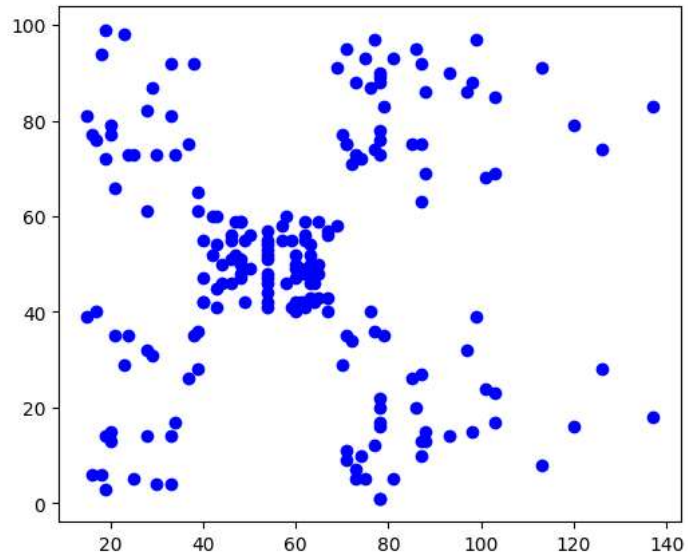
dtype: int64

```
# Select Annual Income and Spending Score as feature variables for clustering and
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print("Features selected for clustering: Annual Income and Spending Score")
```

```
Features selected for clustering: Annual Income and Spending Score
```

```
# Visualize the original data points by plotting Annual Income against Spending Score
plt.figure(figsize=(6,5))
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)'],
    color='blue'
)
```

```
<matplotlib.collections.PathCollection at 0x7f281fe1e270>
```




```
# Apply StandardScaler to normalize the selected features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Feature scaling complete using StandardScaler")
```

```
Feature scaling complete using StandardScaler
```

```
# Initialize the K-Means algorithm with 5 clusters and train it on the scaled feat
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans.fit(X_scaled)
print("Model trained successfully")
```

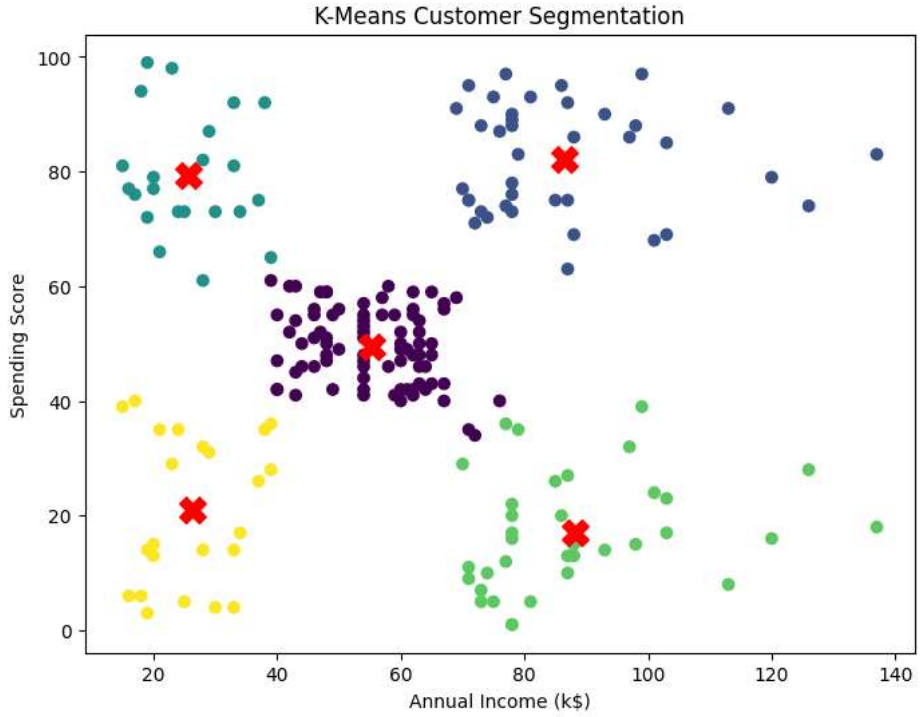
```
Model trained successfully
```

```
# Assign cluster labels to the original dataset and display the first 5 rows with
labels = kmeans.labels_
df['Cluster'] = labels
df[['Annual Income (k$)', 'Spending Score (1-100)', 'Cluster']].head()
```

	Annual Income (k$)	Spending Score (1-100)	Cluster
0	15	39	4
1	15	81	2
2	16	6	4
3	16	77	2
4	17	40	4

```
# Visualize the clusters by plotting Annual Income against Spending Score, with co
plt.figure(figsize=(8,6))
plt.scatter(
X['Annual Income (k$)'],
X['Spending Score (1-100)'],
c=labels,
cmap='viridis'
)
centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
```

```
centers[:,0],
centers[:,1],
c='red',
s=200,
marker='X'
)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.title("K-Means Customer Segmentation")
plt.show()
```




```
# Calculate and display model evaluation metrics: Silhouette Score, WCSS (Inertia),
score = silhouette_score(X_scaled, labels)
print("Silhouette Score:", score)
wcss = kmeans.inertia_
print("WCSS (Inertia):", wcss)
db_score = davies_bouldin_score(X_scaled, labels)
print("Davies-Bouldin Index:", db_score)
```

```
Silhouette Score: 0.5546571631111091
WCSS (Inertia): 65.56840815571681
Davies-Bouldin Index: 0.5722356162263352
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
from sklearn.metrics import davies_bouldin_score
```

```
df=pd.read_csv('/content/Mall_Customers.csv')
print("Data loaded succefully")
```

```
Data loaded succefully
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
df.shape
```

```
(200, 5)
```

```
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	0	19	15	39
1	2	0	21	15	81
2	3	1	20	16	6
3	4	1	23	16	77
4	5	1	31	17	40

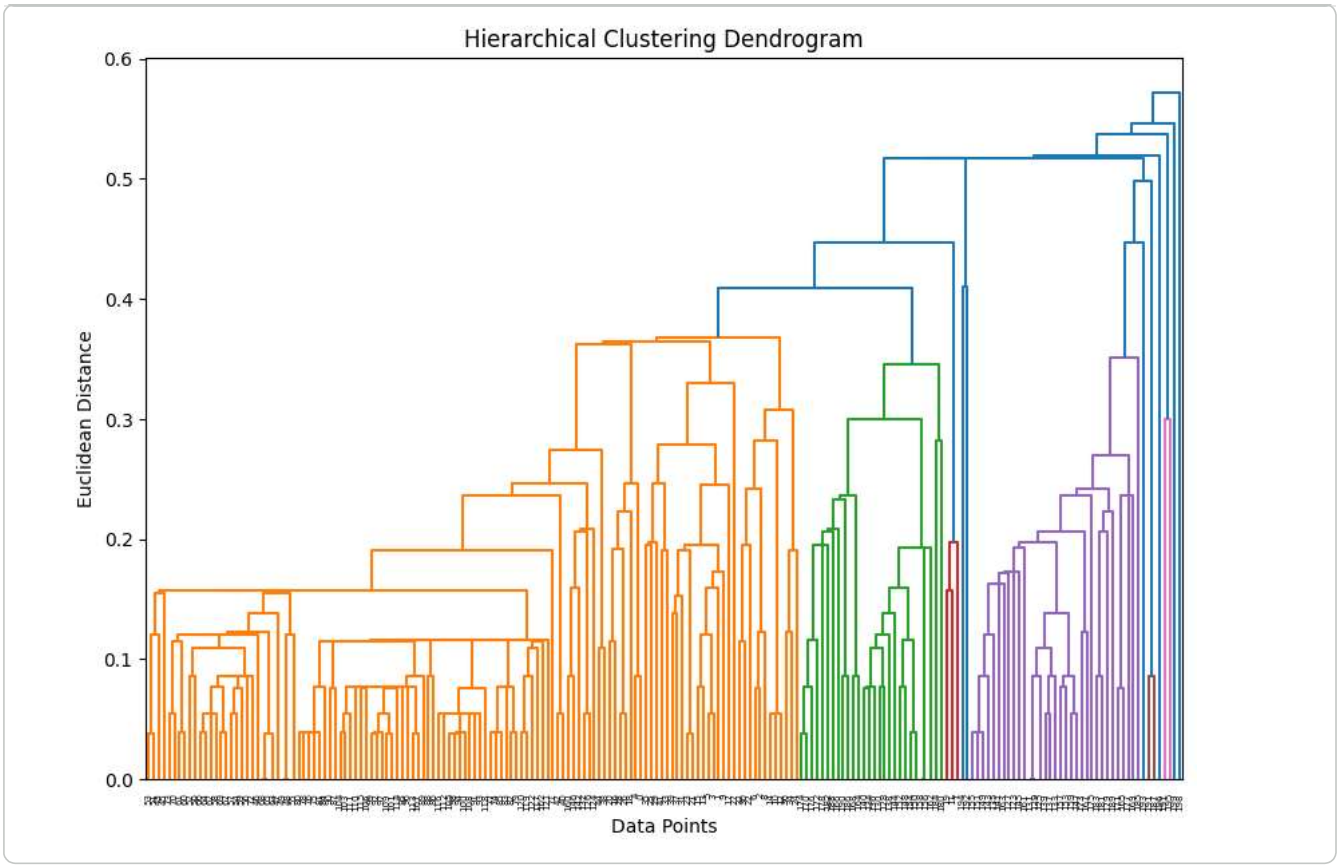
```
# Select Annual Income and Spending Score as feature variables for clustering and d
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print("Features selected for clustering: Annual Income and Spending Score")
```

```
Features selected for clustering: Annual Income and Spending Score
```

```
# Apply StandardScaler to normalize the selected features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Feature scaling complete using StandardScaler")
```

```
Feature scaling complete using StandardScaler
```

```
#plot dendrogram
linked = linkage(X_scaled, method='single')
plt.figure(figsize=(10, 7))
dendrogram(linked, orientation='top', distance_sort='descending', show_leaf_counts=True)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Data Points')
plt.ylabel('Euclidean Distance')
plt.show()
```


Hierarchical Clustering Dendrogram
Euclidean Distance
0.0
0.1
0.2
0.3
0.4
0.5
0.6
Data Points